

Client Questionnaire – RaadiAll Bus & Parcel Booking System

1. Scope & Objectives

- What is the expected initial user base size (daily active users and parcels per day)?
- Do you plan to expand beyond Somalia later (regional scaling)?
- Are there any legal or regulatory approvals required for transport or parcel services in Somalia?
- Should the system support multiple bus operators or just your own company?

2. Functional Modules

- Passenger Website: Should it support guest booking, or must users register first?
- Admin Panel: What kind of analytics dashboards are expected (KPIs, reports)?
- Operator Panel: Will each operator manage their own fleet, revenue, schedules, and staff?

3. Parcel Booking & Tracking

- What is the maximum parcel weight and size supported?
- How should GPS tracking be implemented? (e.g. via Google Maps API or another service)
- Do parcels require insurance or declared value fields?
- How will parcel ID/QR codes be generated — central system or by operator?
- Will parcel pickup and drop points be fixed stations or flexible (door-to-door)?

4. Payments & Billing

- Will the payment gateways be integrated directly or through an aggregator API?
- How will refunds and cancellations be handled?
- Do you require multi-currency support or only Somali Shilling (SOS)?

5. Security & Liability

- What specific features should be included in the Security Dashboard?
(e.g. login attempts log, user access logs, blocked users, active sessions, suspicious activity alerts)
- What kind of Fraud Detection rules should be applied?
(e.g. 5 failed logins in 5 minutes → temporary block, new device + high-value action → require MFA)
- Who will be liable for:
 - ❖ User data breaches
 - ❖ Parcel loss/damage
 - ❖ Payment fraud
 - ❖ Late delivery or missed trips.
- Should the system include audit logs of all admin/operator actions?

6. Infrastructure & Performance

- What kind of server environment should be used? (Cloud server, VPS, or local on-premises server)
- Any specific devices to test on (common phones used in Somalia)?
- Do you need accessibility features (screen reader, high contrast)?
- Are you okay with using third-party libraries for speed, or only custom-built?

7. Timeline & Deliverables

- What is the acceptance criteria for each phase's delivery?

8. Support & Maintenance

- How will support tickets be raised (email, system portal)?
- What is considered a “minor update” under free support?
- Will the client provide server/hosting costs during the support phase?

Your Chosen Stack

- **Mobile Frontend:** Flutter
- **Web Frontend:** React.js / Vue.js
- **Backend:** Node.js with NestJS framework
- **Database:** PostgreSQL

Our Chosen Technology Stack

- **Mobile Frontend:** React Native
- **Web Frontend:** Vite with React
- **Backend:** Express (Node.js)
- **Database:** MongoDB

Why I Chose This Stack

1. Mobile Frontend — React Native

React Native allows us to build **both iOS and Android apps from one single codebase**. It uses real native components, so the performance feels close to a fully native app. With one team writing JavaScript/TypeScript, we can create and update features on both platforms at the same time.

Benefits:

- Faster development and updates
- Lower cost (one team, one codebase)
- Easier long-term maintenance
- Native-like performance and user experience

2. Web Frontend — Vite with React

Vite is a modern build tool that makes development **very fast** and gives users a **smooth, fast-loading website**. It works perfectly with **React**, which is one of the most popular and widely supported web frameworks. This helps us build powerful and dynamic web interfaces quickly.

Benefits:

- Instant feedback during development (hot reload)
- Very small, optimized production builds
- Faster page loading and better user experience
- Large community and reusable components

3. Backend — Express (Node.js)

Express is a simple and flexible backend framework for **Node.js**. It is lightweight compared to NestJS, so we can build APIs and features **faster and with more control**. It also has a huge ecosystem of ready-made libraries, which saves time and effort.

Benefits:

- Fast development speed
- Easy to customize and scale
- Well-supported and widely used
- Works perfectly with JavaScript/TypeScript stack

4. Database — MongoDB

MongoDB is a NoSQL database that stores data in a flexible JSON-like format. Unlike rigid relational databases such as PostgreSQL, it lets us **change or add fields easily** as business needs grow. It is highly scalable and integrates naturally with JavaScript-based backends.

Benefits:

- Flexible data model for fast feature changes
- Faster development without complex migrations
- Easy horizontal scaling as users grow
- Works seamlessly with Node.js and Express

Overall Advantage

This stack uses **one common language (JavaScript/TypeScript)** across mobile, web, and backend. That makes the project **faster, cheaper, and easier to maintain**. It also allows us to deliver new features quickly while keeping performance and quality high.